

Fullspace 实战

Capability-Manifold Agent Runtime

杜乐艺

让 AI 智能体自己学会 “下一步该做什么”

从零开始，用 “找最像的能力” 取代画流程图，编排你的智能体

基于 Fullspace 0.1.0 · pip install fullspace · 2026

前言

如果你从来没接触过“智能体（agent）”，也没关系——这本书就是写给你的。我们会从“一个 agent 到底是什么”讲起，一个词一个词地解释，不假设你懂任何专业术语。只要你愿意跟着把代码敲一遍，就能学会。

Fullspace 是一个 Python 工具库（可以理解为别人写好的一箱工具，你拿来就能用）。它的作用是：帮你的 AI 程序**自动决定下一步该干什么**。比如用户问“怎么退款”，程序能自己判断该去查订单、还是转人工、还是直接回答。

市面上做这件事的工具，大多要求你提前画好一张“流程图”（先做 A，再做 B，遇到情况就走 C……）。Fullspace 的不同之处在于：它**不画流程图**。它把每件能干的事看成一个“能力”，当用户提出需求时，让程序去“找最像的那个能力”来执行。这个听起来简单的改变，会带来很多好处，后面会——展开。

一句话理解整本书

把“能力”想象成一座图书馆里的书，每本书都贴了一组“数字标签”。用户的问题也变成一组数字标签，程序只要找“标签最像的那本书”就知道该干什么了——这就是 Fullspace 的核心，剩下的都是细节。

你需要什么基础

- 会一点点 Python 就行：能看懂 def、字典、列表，就够了；
- 不需要懂高等数学，不需要懂神经网络；
- 有一台装了 Python 3.10 及以上版本的电脑（第 4 章教你怎么装）；
- 最好用过 ChatGPT 这类对话工具——知道“大语言模型”是个能聊天的 AI 就行。

怎么读这本书

第 1 章是**给零基础读者的预热**，用大白话把后面要用到的词（智能体、大模型、向量、检索……）全讲一遍，强烈建议先读。第 2~3 章讲“为什么要用 Fullspace”和“它长什么样”。第 4 章带你装好、跑通第一个例子。第 5~11 章是核心机制，一章讲一个部件，每章都会重复“这是什么、为什么需要、怎么用”。第 12 章是**场景实战**，用四个真实业务场景把前面学的串起来。第 13 章是性能对比，第 14 章讲怎么扩展。书末附录有示例索引、API 速查和术语表。

书里所有代码都来自 Fullspace 0.1.0 的真实功能，并且大多可以直接运行——不需要任何 API 密钥、不需要花钱。代码清单前面会有“清单 N-M”的编号，方便你对照。

目 录

你需要什么基础	3
怎么读这本书	4
1.1 什么是“智能体” (agent)	10
1.2 什么是“大语言模型” (LLM)	11
1.3 最关键的概念：把文字变成数字 (向量 / 嵌入)	11
1.4 什么是“最近邻”和“检索”	12
1.5 什么是“框架”和“pip”	12
1.6 一个贯穿全书的类比	13
1.7 小结	13
2.1 先看看“画流程图”是怎么做的	15
2.2 画流程图的三个麻烦	15
2.3 Fullspace 的不一样：“找最像的能力”	17
2.4 两件要记住的事	17
2.5 小结	18
3.1 一句话概括	19
3.2 几大部件各管什么	20
3.3 智能体“跑一圈”经历哪些步骤	20
3.4 Fullspace 在什么情况下更快	21
3.5 小结	22
4.1 安装	23

4.2 第一个智能体	24
4.3 看懂结果: trajectory 和 terminated_by	26
4.4 让“找最像”变聪明(可选)	26
4.5 把能力画成一个 3D 球	27
4.6 小结	27
5.1 Capability: 一本书	28
5.2 把描述变成数字: HashEmbedder	29
5.3 怎么算“像不像”: 余弦相似度	30
5.4 查找的两种实现: 笨办法 vs 聪明办法	30
5.5 给人看的 3D 地图: Projector	31
5.6 把一切串起来: Manifold	31
5.7 最有趣的能力: 找不到就“长”一个出来	32
5.8 小结	32
6.1 翻完一本书后, 可以留三种“走向指令”	34
6.2 写书的人很自由: 返回什么都可以	35
6.3 四种启动方式	36
6.4 管理员跑一圈, 具体做了什么	36
6.5 记事本怎么更新: reducer	37
6.6 为什么会停: 8 种原因	37
6.7 小结	38
7.1 什么是“流动策略”	39
7.2 模式一: 一次一本 (DiscreteFlow)	39

7.3 模式二：一次翻一批 (FieldFlow)	40
7.4 模式三：越翻越多 (WavefrontFlow)	40
7.5 三种模式怎么选	41
7.6 小结	41
8.1 三层判断，层层把关	43
8.2 第一层：够像就直接用	44
8.3 第二层：实在分不清，才请外援	44
8.4 第三层：找不到就造一本	45
8.5 给查找加个缓存：少花冤枉钱	45
8.6 小结	45
9.1 记事本里每条记录，可以有不同的合并规则	47
9.2 存档：检查点 (Checkpoint)	48
9.3 两种存档位置：内存 vs 硬盘	48
9.4 存档、回档、穿越	49
9.5 记事本还记了“走过的路线”	49
9.6 小结	50
10.1 为什么要打通	51
10.2 把别人的子流程，当成一本书	52
10.3 把自己变成别人的一个零件	52
10.4 变成通用积木：Runnable	53
10.5 小结	53
11.1 每翻一本书，报告一次	54

11.2 同步地看进度	55
11.3 异步地跑, 配合真实大模型	55
11.4 小结	55
12.1 为什么要看场景	57
12.2 场景一: 智能客服——把用户问题自动分发给对的能力	58
12.3 场景二: RAG 文档问答——查资料、再总结、再回答	59
12.4 场景三: ReAct 多步推理——想想、做做、看看, 循环到解决	60
12.5 场景四: 遇到没见过的需求, 自动“长”出能力	62
12.6 场景串讲: 这些场景背后的共同点	64
12.7 小结	64
13.1 怎么对比才公平	65
13.2 六个对比任务	66
13.3 老实说: 简单任务上, Fullspace 反而多干活	66
13.4 能力很多时: Fullspace 大幅领先	67
13.5 怎么自己跑这份评测	67
13.6 小结	68
14.1 换一种“文字变数字”的方法	69
14.2 换一种查找方法	70
14.3 换一种“翻书节奏”	71
14.4 换一种存档方式	71
14.5 小结	72
D.1 一个浪费: 纯 intent 每跳都做一次 ANN	79

D.2 混合路由：确定走 goto，语义走 intent	80
D.3 再加一层：决策缓存	80
D.4 实测：8 个 agent 的 K12 教育对比	81
D.5 下一步：让框架自己学	82

第 1 章

零基础起步：先把几个词搞懂

这一章不写代码，只聊天。读完它，你就能看懂后面所有的内容。如果你已经熟悉这些概念，可以跳到第 2 章。

1.1 什么是“智能体” (agent)

先说一个大白话的定义：**智能体 (agent) 就是一个能自己分好几步去完成任务的 AI 程序。**

举个例子。你让一个智能体“帮我查一下今天的天气，如果是雨天就提醒我带伞”。它会这么做：第一步，去网上查天气；第二步，看看是不是雨；第三步，如果是，给你发提醒。它不是一次性回答你，而是**自己拆成几步、自己决定每步干啥**。这就是智能体和普通聊天机器人的区别——普通机器人只能一问一答，智能体能“自己走流程”。

术语：智能体 / agent

agent 这个词来自英文，意思是“代理、行动者”。在 AI 里，它特指“能自主完成多步任务的程序”。中文常叫“智能体”。你可以把它想成一个能自己干活的小助手。

那么问题来了：这个智能体**怎么知道每一步该干什么**？这正是 Fullspace 要解决的核心问题。在回答它之前，我们再认识几个词。

1.2 什么是“大语言模型”（LLM）

你可能用过 ChatGPT、文心一言、通义千问这类工具。它们背后的技术叫**大语言模型（Large Language Model，简称 LLM）**。简单说，它是一个“读了很多很多文字、所以能理解和生成人类语言”的 AI。

在 Fullspace 里，大语言模型不是必须的——我们前几章的例子都不用它，照样能跑。但在真正的产品里，大语言模型通常扮演“大脑”：由它来读懂用户的话、决定下一步、生成回答。Fullspace 负责把“大脑”的每一步安排好。

术语：LLM / 大语言模型

LLM 是 Large Language Model 的缩写。可以理解为一个“能读能写人类语言”的 AI 大脑。本书例子里用不到它也能跑；等你接了真实产品，再把它接进来。

1.3 最关键的 concept：把文字变成数字（向量 / 嵌入）

这是全书最重要的一个概念，请耐心看。电脑其实看不懂文字，它只会算数字。所以我们要想办法**把一句话变成一串数字**。这串数字就叫**向量（vector）**，这个“变成数字”的过程叫**嵌入（embedding）**。

生活类比：图书馆的数字标签

想象图书馆里有几千本书。图书管理员给每本书贴了一张卡片，卡片上是 256 个数字，比如 [0.2, -0.5, 0.8, ...]。这些数字不是随便编的——**内容越像的两本书，它们的数字也越像**。讲“数学”的两本书，数字会很接近；一本讲数学、一本讲做菜，数字就差很远。这串数字就是“向量”，给书贴数字卡片的过程就是“嵌入”。

为什么要这么做？因为一旦文字变成了数字，电脑就能用数学方法比较“两句话有多像”——比如算它们数字的差。这样，“找最相关的那条”就变成了一个简单的数学题。

术语：向量 / 嵌入 / embedding

向量 (vector)：一串代表文字“含义”的数字，比如 256 个数。嵌入 (embedding)：把文字变成向量的过程。你可以把向量理解成一句话的“数字身份证”——身份证号越接近的两个人，越可能是同类。

1.4 什么是“最近邻”和“检索”

有了向量，我们就能做一件很有用的事：**最近邻查找**。它的意思是——在一大堆东西里，找出和目标“数字最像”的那一个或几个。

还是图书馆的例子。你问了一个问题“怎么做番茄炒蛋”，这个问题也被变成一串数字。然后程序在所有书里找：哪本书的数字和我的问题最像？找到那本“番茄炒蛋菜谱”，就是“最近邻”。这个“找”的过程，也叫**检索 (retrieval)**。

术语：最近邻 / ANN / 检索

最近邻 (nearest neighbor)：数字最像的那个。ANN (Approximate Nearest Neighbor, 近似最近邻)：书太多时，用聪明的方法快速“近似”找到最像的，而不是一本本比（一本本比叫“暴力”，很慢）。检索 (retrieval)：泛指“找出来”这个动作。

Fullspace 的核心，说白了就是：把“能力”变成书，给每本能力贴上数字标签；用户的问题也变成数字标签；然后做一次最近邻查找，就知道该用哪个能力了。

1.5 什么是“框架”和“pip”

框架 (framework) 就是别人写好的一套半成品代码，帮你省去重复劳动。Fullspace 就是一个框架——它已经帮你写好了“找能力、跑步骤、存状态”这些麻烦事，你只要告诉它“我有哪些能力、每个能力干什么”，就能用。

pip 是 Python 自带的小工具，用来从网上下载并安装别人写好的代码包。一行命令就能把 Fullspace 装到你的电脑上：

```
pip install fullspace
```

清单 1-1 安装 Fullspace

装好之后，你就能在自己的代码里 `import fullspace` 来用它了。Fullspace 已经发布到 Python 官方的包仓库（叫 PyPI），所以上面这行命令谁都能直接用。

1.6 一个贯穿全书的类比

我们把前面几个词串起来，用一个故事帮你在脑子里建起画面：

图书馆类比（请记住它）

你开了一家“万能图书馆”。馆里每本书 = 一个“能力”（能干的一件事，比如“查天气”“算数学”“翻译”）。每本书都贴了数字标签（向量）。顾客进门提需求（比如“我要算账”），需求也变成数字标签；图书管理员（Fullspace）看一眼，找到数字最像的那本书（最近邻 / 检索），翻开来执行。执行完，管理员再根据结果，去找下一本最像的书……直到问题解决。整本书，讲的都是怎么把这家图书馆开好。

如果你把上面这段话读懂了，你已经理解了 Fullspace 80% 的设计。后面我们只是把这个故事里的每个环节，用代码和更精确的术语展开。

1.7 小结

这一章我们认识了五个词：智能体（能自己分步干活的 AI 程序）、大语言模型（能理解语言的大脑）、向量/嵌入（把文字变成数字）、最近邻/检索

（找数字最像的那个）、框架/pip（别人写好的工具 + 安装工具）。还建立了一个贯穿全书的“图书馆”类比。接下来，我们看看 Fullspace 为什么要用这种方式，而不像别人那样画流程图。

第 2 章

从画流程图到找最像的：思路的大转变

大多数 AI 工具要求你画一张“先做 A 再做 B”的流程图。Fullspace 偏不。这一章用大白话讲清楚：为什么画流程图会遇到麻烦，而“找最像的能力”能避开这些麻烦。

2.1 先看看“画流程图”是怎么做的

假设你要做一个客服机器人。用传统思路，你会画一张图：用户提问 → 判断是“退款”还是“咨询” → 如果退款就走退款流程，如果咨询就走咨询流程 → 最后给出回答。这张图是用**节点（圆圈，表示一个步骤）**和**边（箭头，表示走向）**连起来的，所以叫“图（graph）”。

这种做法很直观，但藏着三个麻烦。

2.2 画流程图的三个麻烦

麻烦	大白话解释	带来的后果
图一旦画好就改不了	你得在上线前把所有分支都想清楚、画出来	运行时遇到没画过的情况，就抓瞎
只能“点菜单”式选择	判断条件只能从你预先写好的几个选项里挑	用户说了句没预料到的话，程序要么报错要么乱答
能力越多越慢	每多一个分支，每次判断都要多比一遍	能力从 10 个涨到 1000 个，反应越来越慢

表 2-1 传统“流程图”做法的三个麻烦

第一个麻烦举个例子：你上线时只画了“退款”和“咨询”两条路。结果某天用户问“怎么开发票”——你的图里没有这条边，机器人就不知道该怎么办了，因为它没法在运行时“长出”一条新路。

第二个麻烦更常见。传统做法里，“判断走哪条路”是一段代码，它只能返回你预先写好的几个名字：

```
def 判断走向(用户问题):
    if 用户问题 == "退款":
        return "退款流程"
    elif 用户问题 == "咨询":
        return "咨询流程"
    # 没写“开发票”？那这里要么报错，要么乱走
```

清单 2-1 传统做法：判断条件只能返回预先写好的名字

术语：节点 / 边 / 条件边

节点 (node)：流程图里的一个圆圈，表示一个步骤。边 (edge)：连接两个节点的箭头，表示“做完这个去做那个”。条件边 (conditional edge)：带判断的箭头——根据情况走不同方向。传统框架（最有名的叫 LangGraph）就是用这套概念工作的。

2.3 Fullspace 的不一样：“找最像的能力”

Fullspace 换了个思路。它**不画箭头**，而是把每件能干的事登记成一个“能力”，给每个能力写一句描述（比如“处理用户的退款申请”）。记住第 1 章的图书馆类比——这就是给书贴数字标签。

```
# 登记 3 个能力（相当于上架 3 本书）
manifold.register_many([
    Capability("算账", "perform arithmetic and math calculations"),
    Capability("搜索", "search the web for information"),
    Capability("结束", "final answer output", metadata={"sink": True}),
])
# 不需要任何 add_edge! 直接把用户问题丢给引擎
res = engine.run("perform math calculations on numbers")
print(res.trajectory)  # ['算账', '结束']
```

清单 2-2 Fullspace: 不用画箭头，靠“找最像”来决定走哪

看，代码里**没有一条“箭头”**。那句英文“perform math calculations on numbers”被变成数字，和所有能力的描述比对，最像的是“算账”——于是它自动走了“算账”。换一句“search the web”，它就会走“搜索”。**用户的话本身就是走向判断。**

这一个改变，是全书所有好处的根源

把“一次次的条件判断”换成“一次找最像的”。就这一个替换，让程序能在运行时遇到新情况也不怕（找最像的就行）、让能力变多也不变慢（找最像的可以用快速算法）、让程序能自己“长出”新能力。后面每一章，都是在讲这个替换带来的具体好处。

2.4 两件要记住的事

第一件：图本身没有“立体感”

把 5 个圆圈画在平面上，和画在一个球面上，其实是同一张图（每个点都能连到其他点）。所以光说“3D 球面”没有意义，必须让“形状”真正影响计算。Fullspace 的做法是：真正的判断在“高维数字空间”里做（一串 256 个数字的空间），你看到的那个漂亮的 3D 球面只是给人看的“地图”，程序判断时根本不用它。

第二件：找最像的，比画箭头强

与其提前画好一堆箭头、每个分叉判断一次，不如用一次“找最像的”来定位。这个理念贯穿全书。

这两点解释了 Fullspace 所有“奇怪”的设计：为什么真正的判断在高维数字里做、为什么那个 3D 球只是好看、为什么能力能自己长出来。后面我们会落到代码里。

2.5 小结

传统做法像画一张固定的流程图，会遇到“改不了、只能点菜单、变多就慢”三个麻烦。Fullspace 换成“找最像的能力”，用一个简单的数学查找，同时解决了这三个麻烦。下一章，我们从高处看看 Fullspace 整体由哪些部件组成。

第 3 章

全景：Fullspace 由哪些部件组成

在动手之前，先从高处看一眼全貌。这一章介绍 Fullspace 的几大部件，以及一个智能体“跑一圈”经历了哪些步骤。

3.1 一句话概括

Fullspace 是一个 Python 库，你只要**装上就能用**，不需要买显卡、不需要申请密钥。它的核心只依赖一个叫 NumPy 的数字计算库（几乎所有 Python 环境都有）。一些高级功能（比如更快的查找、真正的语义理解）是可选的，需要时再额外装。

为什么默认这么轻

Fullspace 的设计哲学是：第一次安装，不该拉进来一堆笨重的东西。所以它自带了三个“够用”的默认部件——用哈希方法把文字变成数字（HashEmbedder）、用笨办法但绝对准确的查找（NumpyAnnIndex）、用简单统计画出的 3D 地图（PCAProjector）。等你要上正式产品，再换成更强的版本。

3.2 几大部件各管什么

部件	管什么（大白话）
manifold（流形）	那家“图书馆”：登记能力、贴数字标签、提供查找
engine（引擎）	那个“图书管理员”：一圈圈地找书、翻书、决定下一本
state（状态）	“记事本”：记住中间结果，还能随时存档、回档
interop（互操作）	“翻译官”：和别的流行工具（LangGraph 等）互相打通
eval（评测）	“考官”：拿 Fullspace 和别的工具对比考试
viz（可视化）	“展示柜”：把能力画成一个可旋转的 3D 球

表 3-1 Fullspace 的几大部件

它们的关系是：manifold 是底层“图书馆”（只提供数据，不主动干活）；engine 是上层“管理员”，调用 manifold 来跑流程；state 帮 engine 记事；interop 让 engine 能和外部工具合作。记住图书馆类比，这些部件的位置就清楚了。

术语：manifold / 流形

manifold 这个词本意是“流形”（一个数学名词，指弯曲的空间）。在 Fullspace 里，你可以直接把它当成“那家图书馆”的名字——所有能力（书）和它们的数字标签（向量）都住在里面。不用纠结数学含义。

3.3 智能体“跑一圈”经历哪些步骤

不管你是“一口气跑完”（run）还是“一步一步看”（stream），背后都是同一个循环。用大白话讲，管理员每跑一圈做这几件事：

顾客提需求 (task)

-> 把需求变成数字，找到最像的第一本书

-> 翻开书执行 (run)

-> 把结果记到记事本上 (merge)

-> 书里写明“接下来想干什么” (intent)

-> 把记事本存个档 (checkpoint, 可选)

-> 根据 intent，找下一本最像的书

-> 到达终点 / 被叫停 / 预算用完 -> 结束

清单 3-1 管理员的一圈（闭环）

术语：闭环 / intent / sink

闭环（closed loop）：一圈接一圈、能自己转起来的流程。intent（意图）：每本书执行完后，留下的“接下来想干什么”的一句话，用来找下一本。

sink（汇点）：一种特殊的“终点书”，翻到它就表示任务完成。

3.4 Fullspace 在什么情况下更快

Fullspace 宣称在“能力很多”的时候更快，原因有三个（第四个还在做）：

快的来源	大白话解释
找一次顶判断多次	一次“找最像的”，顶得上传统做法里好多次条件判断
查找本身能加速	书很多时，有聪明算法（FAISS）让查找快几十上百倍
能同时翻好几本书	可以一步激活好几个相近的能力，不用排队等

表 3-2 Fullspace 快在哪

这三点分别对应后面的第 8 章（路由器）、第 5 章（查找索引）、第 7 章（流动策略）。现在不用记，知道“快是有道理的”就行。

3.5 小结

Fullspace 由 manifold（图书馆）、engine（管理员）、state（记事本）、interop（翻译官）等部件组成。管理员每跑一圈，就是“找书→翻书→记事→找下一本→到终点”的循环。它快的根源是“找最像的”这种做法。下一章，我们真正动手，装好 Fullspace、跑通第一个例子。

第 4 章

动手：装好 Fullspace，跑通第一个例子

十分钟，从零到跑通你的第一个智能体，看懂它走了哪几步。全程不需要任何密钥。

4.1 安装

Fullspace 已经发布到 Python 官方包仓库 PyPI，版本 0.1.0。只要你的电脑装了 Python 3.10 及以上，一行命令就能装好：

```
pip install fullspace
```

清单 4-1 安装 Fullspace（已发布 PyPI）

装完验证一下，能打印出版本号就说明成功了：

```
python -c "import fullspace; print('Fullspace 已就绪')"
```

清单 4-2 验证安装

想要更多能力时（可选）

`pip install faiss-cpu` 可以让查找变快（能力很多时）；`pip install sentence-transformers` 可以让“找最像”更聪明（真正理解语义，而不只是看字面）。新手先不用装，默认的就够学。

术语：PyPI / pip / 包

PyPI 是 Python 官方的“应用商店”，存放大家共享的代码包。`pip` 是从 PyPI 下载安装的工具。包（package）就是别人写好、打包好、能直接用的代码。Fullspace 就是一个包。

4.2 第一个智能体

下面这段代码是本书第一个能跑的例子。它建了 3 个能力（搜索、总结、结束），给每个能力绑定了“该干什么活”，然后运行。你可以直接复制运行：


```
from fullspace import Capability, HashEmbedder, Manifold
from fullspace.engine import Engine, NodeResult

# 1) 建图书馆，登记 3 本书（能力）
manifold = Manifold(HashEmbedder())
manifold.register_many([
    Capability("search", "search the web for information"),
    Capability("summarize", "summarize a long document into key points"),
    Capability("end", "final answer output", metadata={"sink": True}),
])

# 2) 请个管理员（引擎），告诉它每本书翻开后干什么
agent = Engine(manifold)
agent.bind("search",
    lambda ctx: NodeResult(updates={"found": "..."},
                               intent="summarize a long document into key points"))
agent.bind("summarize",
    lambda ctx: NodeResult(updates={"summary": "..."}, goto="end"))
agent.bind("end",
    lambda ctx: NodeResult(updates={"answer": "..."}))

# 3) 把顾客需求丢给管理员
result = agent.run("search the web for information")
print(result.trajectory)  # ['search', 'summarize', 'end']
```

清单 4-3 第一个 Fullspace 智能体

逐行解释这段代码

第 1 步：Capability(名字, 描述) 就是登记一本书，描述用来贴数字标签；
metadata={"sink": True} 标记“结束”是终点书。第 2 步：bind(能力名, 函数) 告诉管理员“这本书翻开时执行这个函数”。函数返回的 NodeResult 里：updates 是要记到记事本的内容；intent 是“接下来想干什么”（软走向，靠找最像）；goto 是“直接去某本”（硬走向）；第 3 步：run(需求) 启动管理员。

运行结果 trajectory (轨迹) 是 ['search', 'summarize', 'end'], 意思是管理员依次翻了“搜索→总结→结束”三本书。注意：search 靠 intent 自动

找到了 summarize（没画箭头！），summarize 靠 goto 直接跳到 end。

4.3 看懂结果：trajectory 和 terminated_by

run 返回的结果里，最常用的几个字段：

字段	是什么意思
state	最终记事本里的全部内容
trajectory	管理员依次翻过哪些书（能力名列表）
steps	一共翻了几次
terminated_by	为什么停下来了（原因）

表 4-1 结果对象的主要字段

terminated_by 有 8 种原因

最常见的是 sink（翻到了终点书，正常结束）和 budget（步数超预算被叫停）。另外还有 halt（主动叫停）、no_intent（没说下一步干啥，自然结束）等。看到 sink 就代表任务顺利跑完了。

4.4 让“找最像”变聪明（可选）

默认的 HashEmbedder 只会比较“字面重叠”——比如 search 和 query 是两个不同的词，它就觉得不相干。这在学习阶段够用，但在真实产品里不够聪明。换成 sentence-transformers，它就能真正“理解”语义：

```
from fullspace.manifold.embedding import SentenceTransformersEmbedder
m = Manifold(SentenceTransformersEmbedder("all-MiniLM-L6-v2"))
```

清单 4-4 换成真正理解语义的嵌入（需先 pip install sentence-transformers）

HashEmbedder 不懂同义词

它只看字面有没有相同的词。这对学习框架原理完全够用（而且快、不要钱、不联网），但生产环境请换神经网络嵌入。本书前 10 章都用 HashEmbedder，你能在自己电脑上立刻跑出结果。

4.5 把能力画成一个 3D 球

Fullspace 自带一个可视化，能把你的能力画成一个可以旋转的 3D 球面：

```
python -m fullspace.viz      # 会生成 fullspace_sphere.html
```

清单 4-5 生成 3D 能力球（用浏览器打开）

它会登记几个能力、跑一次流程，然后把能力和走过的轨迹画在一个半透明球面上。用浏览器打开那个 html 文件就能拖动旋转。但要牢记本章最重要的一句话——

球只是给人看的地图，程序判断不用它

真正的判断在“256 个数字的高维空间”里做，那个 3D 球只是把高维空间投影成你能看懂的形状。程序找书时，用的是高维数字，不是球面上的位置。这一点在全书的代码和测试里都严格保证。

4.6 小结

你已经会装 Fullspace (pip install fullspace)、跑通第一个智能体、看懂 trajectory 和 terminated_by、知道怎么升级到更聪明的嵌入和 3D 可视化。下一章我们下沉到底层，看“图书馆”内部是怎么把能力变成数字、怎么查找的。

第 5 章

图书馆内部：能力怎么变数字、怎么查找

这一章打开 manifold（图书馆）的盖子，看它如何把一句描述变成数字、如何快速找出最像的能力。每个部件都用大白话讲一遍原理。

5.1 Capability：一本书

```
Capability(  
    id="search",                # 名字（编号）  
    description="search the web", # 描述（用来变数字标签）  
    metadata={"sink": False},    # 附加信息  
    vector=None,                # 数字标签，登记时自动填  
)
```

清单 5-1 一个能力（书）长什么样

“终点”只是一个标记，不是特殊类型

Capability 没有“终点书”这种特殊种类。任何一本书，只要在 metadata 里写上 {"sink": True}，就成了终点书。对比传统做法要专门设一个 END 节点——Fullspace 把“终点”降维成了一个标记，简单多了。

注意 vector 一开始是空的 (None)。一本书在“还没被图书馆接收”之前，是没有数字标签的。标签在登记 (register) 时由图书馆自动贴上。

5.2 把描述变成数字：HashEmbedder

默认的 HashEmbedder 用了一个叫“特征哈希”的老办法。原理用大白话讲：把描述拆成一个个词，每个词通过一个固定的数学函数 (md5) 算出两件事——**落到第几格** (桶号)、**加还是减** (符号)。最后把所有词的效果累加，归一化，得到一串数字。

```
def embed(self, text):
    vec = 全零向量(256 维)
    for 词 in 拆词(text):
        摘要 = md5(词)
        桶号 = 摘要前4字节 % 256      # 这个词落到第几格
        符号 = +1 或 -1              # 这个词贡献正还是负
        vec[桶号] += 符号
    归一化(vec)
    return vec
```

清单 5-2 HashEmbedder 的核心逻辑 (简化)

为什么这样能比较“像不像”

同一个词永远落同一个格、永远同正同负 (因为 md5 是确定的)。所以两句话**共有的词越多，它们在同一格里累加的方向越一致，最后算出来的数字就越像**；没有关系的词落点是随机的，会互相抵消。于是“数字的相似度” $\approx$ “用词的重叠度”。

为什么用 md5 而不是 Python 自带的 hash

Python 自带的 hash 每次启动程序都会变 (出于安全)，会导致同一句话在不同运行里数字不一样，图书馆就乱套了。md5 是固定的，保证任何时候同一句话都是同一串数字。

5.3 怎么算“像不像”：余弦相似度

有了两串数字，怎么判断它们像不像？最常用的办法叫**余弦相似度**（**cosine**）。你不用懂公式，只要记住：**它算出来是 -1 到 1 之间的一个数，越大越像**。1 表示完全一样，0 表示没啥关系，-1 表示刚好相反。

术语：余弦相似度 / cosine

把两串数字看成两个箭头，余弦相似度就是看这两个箭头“方向有多接近”。方向一致（同向）就是 1，垂直就是 0，反向就是 -1。它只看方向不看长短，所以很适合比较“含义”。

Fullspace 里还有个 `top_k` 函数，负责“从一堆相似度里挑出最大的 `k` 个”。它用了个叫 `argpartition` 的聪明办法——不把全部排序（那很慢），而是快速圈出前几名，再只对这几名排序。当能力很多、只要前几个时，这能省很多时间。

5.4 查找的两种实现：笨办法 vs 聪明办法

`NumpyAnnIndex` 是“笨办法”：你要找最像的，它就把你的问题和**每一本书**都比一遍，挑最像的。书少的时候没问题，准确、简单；但书多了（比如上万本）就慢。

笨办法里的小聪明：脏标记

登记新书时不马上重新整理书架，而是插个“脏”小旗；只有真要查的时候，且看到旗子是脏的，才整理一次。因为查找远多于登记，这个小优化让整理次数大大减少——这是“读写不均衡”时的经典优化。

`FaissIndex` 是“聪明办法”（需要额外装 `faiss-cpu`）。它会把书分门别类放在不同区域（倒排索引），查的时候先定位到大致区域，再在小区里找——书很多时能快几十上百倍。它还有个细节：书太少时（少于约 3900 本）它会自动退回笨办法，因为那时候分区域反而更慢。

什么时候用什么

学习、调试、能力不多：用默认的 NumpyAnnIndex（笨但准）。上正式产品、能力成千上万：装 faiss-cpu，换 FaissIndex。Fullspace 会让你用一行代码切换，业务代码不用改。

5.5 给人看的 3D 地图：Projector

前面说真正的判断在 256 维数字空间里做，但人脑想象不出 256 维。所以 Fullspace 用一个叫 PCA 的方法，把这 256 维“压扁”成 3 维，画成你能看的球面。

术语：PCA / 投影

PCA（主成分分析）是一种“降维”方法：找出数据里变化最大的几个方向，把高维压到低维，尽量保留原本的结构。投影（projection）：把高维点映射到低维的过程。Fullspace 的投影只为了给人看，**程序判断时绝不读取 3D 坐标**。

PCA 的一个局限：它只保留“全局的大趋势”，可能让原本接近的两本书在 3D 上看着很远。想要更好的局部效果，可以换 UMAP（更聪明但更慢、要额外装）。

5.6 把一切串起来：Manifold

Manifold 就是那家“图书馆”本身，把上面这些部件协调起来。登记一本书（register）时，它同步做四件事：贴数字标签、存进字典、更新书架索引、标记 3D 地图需要重画。

```
def register(self, capability):
    数字 = self.embedder.embed(capability.description) # 贴标签
    capability.vector = 数字
    self._caps[capability.id] = capability # 存字典
    self.index.add(capability.id, 数字) # 更新书架
    self._projection_dirty = True # 地图要重画
    return capability
```

清单 5-3 登记一本书时发生的事（简化）

查找走 `nearest` 方法：它返回最像的几个能力，连带着相似度分数。你只管把需求丢进去，图书馆自己找。

5.7 最有趣的能力：找不到就“长”一个出来

`Manifold` 有个 `find_or_materialize` 方法，实现三条路：

```
hits = self.nearest(需求)
if hits and hits[0].score >= 阈值:
    return hits[0] # 路 A: 够像，直接用
if 没有造书匠:
    return hits[0] if hits else None # 路 B: 没匠人，将就用最像的
新书 = 造书匠(描述, 分数) # 路 C: 现场造一本新书
self.register(新书)
return Hit(新书, 1.0)
```

清单 5-4 三条路：够像就用 / 不够就将就 / 找不到就造一个

这就是“涌现”——程序自己长出新能力

当用户提了个现有能力都不太匹配的需求，传统做法会报错；Fullspace 可以“现场造一个新能力”加进图书馆。这就像图书馆发现顾客老要某类没有的书，就当场印一本上架。这是 Fullspace 区别于传统框架最厉害的一点。

5.8 小结

图书馆 (manifold) 内部：能力是书，HashEmbedder 给书贴数字标签，余弦相似度算“像不像”，NumpyAnnIndex (笨办法) 或 FaissIndex (聪明办法) 负责查找，PCA 把高维压成 3D 给人看。Manifold 把这些串起来，还能在找不到时“长”出新能力。下一章看管理员 (引擎) 怎么一圈圈跑流程。

第 6 章

管理员怎么跑流程：引擎 Engine

Engine 就是那个一圈圈找书、翻书、记事、决定下一本的管理员。这一章拆解它跑一圈的每一步，并解释每本书执行后留下的“三种走向指令”。

6.1 翻完一本书后，可以留三种“走向指令”

每本书（能力）执行完后，会返回一个 `NodeResult`，告诉管理员“下一步去哪”。有三种走向，按优先级从高到低是 **halt > goto > intent**：

```
NodeResult(  
  updates={"found": "..."},           # 记到记事本的内容  
  intent="summarize the findings",    # 走向1：软走向（找最像的）  
  goto="end",                         # 走向2：硬走向（直接跳某本）  
  halt=False,                        # 走向3：强制叫停  
)
```

清单 6-1 `NodeResult`：状态更新 + 走向指令

指令	怎么走	大白话	对应传统做法
intent	把这句话变数字，找最像的能力	我说想干啥，你帮我找	条件边（智能版）
goto	直接跳到指定名字的能力	我点名要这本	固定箭头
halt	立刻停止	不干了，结束	直接到终点

表 6-1 三种走向指令

三个都不写，就自然结束

如果一本书执行完，既没说 intent、也没说 goto、也没 halt，管理员就认为“这步干完了，没有下一步”，于是自然结束（terminated_by = no_intent）。这其实就是“把某个能力当终点”的写法——什么都不返回就行。

6.2 写书的人很自由：返回什么都可以

为了让写代码的人省事，引擎很宽容——你返回的东西会被自动归一化成 NodeResult：

你返回	引擎理解为
什么都不返回（None）	到此为止（halt）
一个字典 dict	只更新记事本，没有下一步（自然结束）
一个 NodeResult	按你说的来
其它乱七八糟的东西	报错

表 6-2 返回值会被自动理解

所以写 handler（每本书翻开时执行的函数）非常随意：只想记事就返回个字典，想停就 return，想完整控制就返回 NodeResult。

6.3 四种启动方式

引擎有四种启动方式，分别对应“同步/异步”和“一口气/一步步看”两个维度：

方式	特点	什么时候用
run	同步，一口气跑完返回结果	最常用，简单任务
stream	同步，但每翻一本就告诉你一次	想看进度、边跑边处理
ainvoke	异步版的一口气跑完	配合异步代码（async）
astream	异步版的逐步看	异步 + 看进度

表 6-3 四种启动方式

run 其实就是 stream 的浓缩

四种方式背后是同一个循环。run 就是把 stream 产生的事件一个个收完，取最后一个。所以它们的行为完全一致，只是“你要不要中途看”的区别。

6.4 管理员跑一圈，具体做了什么

用大白话+伪代码看一圈：

```

当前要翻的书 = flow.select(图书馆, 需求)    # 找本/几本
for 每本书 in 当前要翻的书:
    结果 = handler(记事本, ...)                # 翻开执行
    记事本 = 合并(记事本, 结果.Updates)        # 记事
    if 结果.intent: 收集意图
    if 结果.halt: 叫停
    if 结果.goto: 记下点名
    步数 += 1
    下一批书 = 根据意图/点名找下一本          # 走向

```

清单 6-2 一圈的伪代码

一次翻多本时，是按顺序翻的

高级模式（第 7 章），管理员可以一次拿好几本书一起翻。但它们其实是按顺序一本本翻、一本本记的（不是真同时），只是语义上算“同一批”。这对理解结果没影响。

6.5 记事本怎么更新：reducer

每本书执行后会把 updates 记进记事本。但“怎么记”有讲究——同一个记事本字段，可以有不同的合并规则，这叫 **reducer**。默认是“后写的覆盖先写的”（overwrite）；也可以设成“累加”（add，适合记聊天历史这种越来越长的内容）。

```
def merge_updates(记事本, 更新, 规则表):
    for 字段, 值 in 更新.items():
        规则 = 规则表.get(字段, 默认覆盖)
        记事本[字段] = 规则(记事本.get(字段), 值)
    return 记事本
```

清单 6-3 合并更新：按字段选规则

术语：reducer / 状态

状态（state）：就是那个“记事本”，一个字典，存着到目前为止的所有结果。reducer：决定“新值怎么和旧值合并”的函数。最常见三种：

overwrite（覆盖）、add（累加，适合列表）、last_value（有新值才覆盖，没有就保留旧的）。

6.6 为什么会停：8 种原因

原因	什么时候
sink	翻到了终点书（最常见，正常结束）
halt	某本书主动叫停
no_intent	一本书没说下一步干啥
budget	翻的次数超过预算（默认 25）
empty	图书馆是空的
no_handler	一本书没绑处理函数
bad_goto	点名要一本不存在的书
no_route	找不到下一本

表 6-4 管理员停止的 8 种原因

新手最常看到的是 sink（顺利跑完）和 budget（步数太多被拦，通常说明你的流程没设好终点）。看到这两个你就知道发生了什么。

6.7 小结

引擎通过 NodeResult 的三种走向（halt>goto>intent）决定“下一步去哪”，用 reducer 合并记事本，有 8 种停止原因。四种启动方式背后是同一个循环。下一章看一个进阶但很有用的功能：管理员能不能一次翻好几本书。

第 7 章

一次翻好几本书：流动策略

传统做法一次只能走一步。Fullspace 可以让管理员一次翻开好几本相近的书——这就是“无屏障并行”的来源。这一章讲三种“翻书节奏”。

7.1 什么是“流动策略”

流动策略 (FlowPolicy) 决定**管理员每一步翻几本书**。听起来是个小决定，但它影响很大：一次只翻一本，就不画箭头的传统流程一样；一次翻好几本，就能并行处理。

术语：流动策略 / FlowPolicy

FlowPolicy 是“翻书节奏”的策略。Fullspace 内置三种：Discrete（一次一本）、Field（一次固定几本）、Wavefront（一次比一次多）。你可以把它理解成管理员的“工作模式”。

7.2 模式一：一次一本 (DiscreteFlow)

最简单的模式：每一步只翻开最像的那**一本书**。这等价于传统的“一步一步走”的流程，只是走向靠“找最像”而不是画箭头。

```
class DiscreteFlow:
    def select(self, 图书馆, 需求):
        return 图书馆.nearest(需求, k=1)[:1]  # 只要最像的 1 本
```

清单 7-1 DiscreteFlow: 每步只翻 1 本

传统流程图其实是这种模式的特例

Fullspace 自我定位是传统框架的“超集”：传统框架能做的（一步步走），用 DiscreteFlow + goto 都能做；但 Fullspace 还能做传统框架做不到的（一次翻多本、运行时长新书）。所以学 Fullspace 不会让你失去任何能力。

7.3 模式二：一次翻一批 (FieldFlow)

每一步翻开固定数量的几本相近的书，它们一起执行、一起记事、再把意图合起来决定下一步。这就是**无屏障并行**。

```
class FieldFlow:
    def __init__(self, width=3, min_score=0.0): ...
    def select(self, 图书馆, 需求):
        hits = 图书馆.nearest(需求, k=self.width)
        hits = [h for h in hits if h.score >= self.min_score]
        return hits or 图书馆.nearest(需求, k=self.width)  # 兜底
```

清单 7-2 FieldFlow: 每步翻 k 本相近的

无屏障 vs 传统并行

传统框架要并行，得等所有分支都完成才能进下一步（有个“同步屏障”）。FieldFlow 在同一步里翻多本书，它们的意图直接加权合并成下一步，不用等——所以叫“无屏障”，更顺滑、更快。

7.4 模式三：越翻越多 (WavefrontFlow)

每一步翻开的书数量越来越多，像水波一样从起点向外扩散。适合“探索”类任务——一开始小范围试，不行就扩大范围。

```
class WavefrontFlow:
    def __init__(self, base_width=2, growth=1, max_width=None): ...
    def select(self, 图书馆, 需求):
        self._t += 1
        k = self.base_width + (self._t - 1) * self.growth    # 越来越多
        if self.max_width: k = min(k, self.max_width)
        return 图书馆.nearest(需求, k=k)
```

清单 7-3 WavefrontFlow：扇形扩散

7.5 三种模式怎么选

模式	适合场景	类比
Discrete（一本）	清晰的一步步流程，入门首选	一个个柜台办手续
Field（固定几本）	多个相近专家一起处理	开会时几个人同时发言
Wavefront（递增）	不确定方向，逐步扩大探索	找东西时先近处找、再远处找

表 7-1 三种模式怎么选

新手用默认的 Discrete 就好。等你遇到“想让多个能力同时贡献”的需求，再试 Field。

7.6 小结

流动策略决定每步翻几本书：Discrete 一次一本（传统等价）、Field 一次一批（无屏障并行）、Wavefront 越翻越多（扩散探索）。默认用 Discrete，需要并行时换 Field。下一章我们深入“找下一本”的核心大脑——混合路由器。

第 8 章

找下一本的大脑：混合路由器

路由器（Router）决定“下一步翻哪本书”。它默认只做一次“找最像”，只有在真的拿不准时才请外援（比如问大模型），在完全找不到时还能现场造一本新书。

8.1 三层判断，层层把关

混合路由器（Router）像个谨慎的图书管理员，找下一本书时分三层判断，前面一层搞定了就不进下一层：

1. 没说下一步要干啥 → 直接结束
2. 找最像的 2 本书看看
3. 默认选最像的那本
4. 如果前两名太接近、分不清 → 请外援帮挑（只在这里花钱）
5. 最像的够像（超过阈值） → 直接用它，不花冤枉钱
6. 一个都不够像、但有造书匠 → 现场造一本新书
7. 实在没办法 → 勉强用最像的

清单 8-1 路由器的三层判断

术语：阈值 / 阈值 margin

阈值 (threshold)：一个“够不够像”的及格线分数，默认 0.3。最像的书分数超过它，就算“够像”，直接用。margin (间隔)：判断“前两名是不是太接近、分不清”的一个小范围，默认 0.15。前两名分数差小于它，说明势均力敌，需要外援。

8.2 第一层：够像就直接用

第一层叫**亲和力裁剪**——最像的那本书，分数只要超过及格线，就直接用。这是 Fullspace 比“每步都判断”的传统做法快的原因：一次查找就定了方向，不用反复评估。

8.3 第二层：实在分不清，才请外援

只有当最像的两本书**分数太接近、难分高下**时，路由器才会请一个“消歧器 (disambiguator)”来帮忙挑。在真实产品里，这个消歧器通常接大语言模型——也就是说，**大模型只在“真的拿不准”时才被调用一次**，而不是每步都叫。这能省很多钱和时间。

```
if (有消歧器
    and 第二名存在
    and (第一名分数 - 第二名分数) < 间隔):
    选中的 = 消歧器(意图, 候选们)  # <- 大模型只在这里被调用一次
```

清单 8-2 只有势均力敌时才请外援

大模型从“每步必请”变成“罕见救场”

传统做法里，判断走向往往每一步都要调一次大模型，又慢又贵。Fullspace 默认一次都不调，只在势均力敌时调一次。这是它“又快又省”的关键设计。

8.4 第三层：找不到就造一本

当所有书都不够像（分数都没过及格线），而你有提供一个“造书匠（materializer）”，路由器会**现场造一本新书**并上架。下一章我们会看到，这正是“程序自己长出新能力”的来源。

```
from fullspace.engine.router import Router
m = Manifold(HashEmbedder())
m.register(Capability("greet", "greet the user hello"))
e = Engine(m, router=Router(
    m, threshold=0.99, # 把及格线设极高，强制“找不到”
    materializer=lambda desc, score: Capability("fallback", desc),
))
e.bind("greet", lambda c: NodeResult(intent="zzzzzzzz unmatched gibberish"))
e.bind("fallback", lambda c: NodeResult(halt=True))
r = e.run("greet the user hello")
print(r.trajectory) # ['greet', 'fallback']
print("fallback in m) # True, 新书被造出来并上架了
```

清单 8-3 物化演示（来自测试用例，可直接运行）

8.5 给查找加个缓存：少花冤枉钱

很多任务里，“接下来想干啥”这句话会反复出现（比如 ReAct 循环里老说 act、observe）。每次都重新算数字标签很浪费。CachedEmbedder 会把算过的记下来，下次直接用——循环场景下能少算几十倍。

为什么缓存绝对正确

因为“把一句话变成数字”是一个纯粹的过程（同样的话永远得到同样的数字），所以缓存永远不会失效，放心用。CachedEmbedder 用起来和原来的完全一样，套一层就行。

8.6 小结

混合路由器三层把关：够像就直接用（默认，一次查找）→ 势均力敌才请外援（大模型罕见救场）→ 找不到就造新书（涌现）。大模型从“每步必请”降级为“罕见救场”，这是又快又省的根源。下一章看记事本怎么存档、怎么回档。

进阶：还能更省——混合路由

「每跳都做一次 ANN」在确定流程上是浪费。优化技巧：确定的流程段用 goto (0 次 ANN)，只在语义分叉用 intent，再把重复的 intent 缓存起来。实测能让路由开销追平传统图、延迟更低，同时保留动态能力。完整做法和实测数据见附录 D。

第 9 章

记事本的存档与回档：状态与检查点

能“存档、回档”是一个靠谱程序的基本要求。这一章讲 Fullspace 怎么记住中间结果、怎么随时存档、怎么从存档继续跑——甚至怎么“穿越”回过去的某一步。

9.1 记事本里每条记录，可以有不同的合并规则

第 6 章提过，记事本（状态 state）的每个字段可以挂不同的合并规则（reducer）。这里把三种内置规则讲清楚：

规则	大白话	适合记什么
overwrite（覆盖）	新的直接替换旧的（默认）	大部分字段，比如当前答案
last_value	只有给了新值才替换，没给就保留	可选更新，某步只想改部分字段
add（累加）	把新值追加到旧值后面	聊天历史、操作日志，越来越长

表 9-1 三种合并规则

add 规则的小贴士

传统做法里，记聊天历史得先建个空列表再往里塞。Fullspace 的 add 规则更省事：第一次直接给它一句话，它会自动包成列表；后面再给，自动追加。

9.2 存档：检查点 (Checkpoint)

“检查点”就是游戏里的**存档**——把当前记事本、走到第几步、翻过哪些书，整个存下来。Fullspace 每翻完一本书都会自动存一次档，所以**任何時候出了问题，都能从最近的存档恢复**。

```
Checkpoint(  
    checkpoint_id="任务1:0002",    # 存档编号（任务名+第几步）  
    thread_id="任务1",             # 哪个任务（一次会话）  
    step=2,                        # 走到第几步  
    state={...},                  # 记事本完整内容  
    trajectory=["work", "work"],   # 翻过哪些书  
    parent_id="任务1:0001",        # 上一档是谁（链表）  
    terminated_by=None,            # 为什么停（没停就是空）  
)
```

清单 9-1 一个检查点存了什么

9.3 两种存档位置：内存 vs 硬盘

InMemoryCheckpoint：存在内存里，程序关了就没了，但特别快，适合学习和测试。**SqliteCheckpoint**：存在硬盘上的一个数据库文件里，程序关了还在，适合正式产品。两者用法完全一样，换一行代码即可。

术语：SQLite / 数据库

SQLite 是一个极轻量、不需要单独安装的数据库，Python 自带。Fullspace 用它把存档写进一个 .db 文件。你可以把 SqliteCheckpoint 想成“把存档写进硬盘文件，下次还能读出来”。

9.4 存档、回档、穿越

有了检查点，就能做三件事。**断点续跑**：任务跑一半被打断（比如步数超预算），下次从最近存档接着跑。**看历史**：列出这个任务所有的存档。**穿越**：跳回任意一步的存档，看看当时的状态，甚至从那里分叉出新任务。

```
from fullspace.state import InMemoryCheckpointter
eng = Engine(m, checkpointter=InMemoryCheckpointter())
# 故意只给 2 步预算，跑到一半被拦
r1 = eng.run("work repeat the processing step",
             state={"n": 5}, thread_id="job1", max_steps=2)
print(r1.terminated_by, r1.trajectory)  # budget ['work', 'work']
# 从最近存档接着跑，直到完成
r2 = eng.resume("job1", task="work repeat the processing step", max_steps=25)
print(r2.terminated_by, r2.trajectory)  # sink ['work', 'work', 'work', 'work', 'end']
# 看历史存档
for cp in eng.history("job1"):
    print("第", cp.step, "步", cp.terminated_by)
```

清单 9-2 断点续跑（来自官方示例，可直接运行）

为什么能用 thread_id

thread_id 就是“这次会话/任务的名字”。给了它 + 装了检查点器，引擎才会自动存档。不给 thread_id，引擎就不存档（很多简单任务不需要存档，省事）。

9.5 记事本还记了“走过的路线”

Fullspace 不只存数据，还存“在图书馆里走过哪些书、按什么顺序”——这叫**轨迹 (trajectory)**。所以回档时，你能看到的不只是当时记了什么，还有当时是怎么走过来的。这个轨迹还能配上 3D 坐标画在球面上，方便调试。

但路线只是给人看的，不影响判断

3D 坐标只用来画图 and 调试。程序判断“下一步去哪”永远靠高维数字和走向指令，绝不读 3D 坐标——这是为了避免“地图反过来影响找路”的混乱。

9.6 小结

Fullspace 用合并规则 (reducer) 决定记事本怎么更新，用检查点 (Checkpoint) 每步自动存档，用内存或硬盘两种存储，用 resume/history 实现“断点续跑、看历史、穿越”。轨迹把走过的路线也存了下来，但只供查看。下一章看 Fullspace 怎么和别的流行工具互相打通。

第 10 章

和别的工具互相打通：互操作

Fullspace 不逼你“二选一”。它能把你已有的别的工具（最出名的是 LangGraph）吞进来当能力用，也能把自己变成别的工具的一个零件。这一章讲怎么打通。

10.1 为什么要打通

你可能在别的项目里已经用了 LangGraph 这类工具，里面有现成的子流程。Fullspace 不让你重写——它可以把那个子流程**当成一本书**嵌进自己的图书馆。反过来，别人用 LangGraph 搭的大系统，也可以把 Fullspace **当成其中一个节点**来用。这种“双向打通”是 Fullspace 能真正落地、而不是只能玩的关键。

术语：LangGraph / 互操作

LangGraph 是目前最流行的“画流程图”式智能体框架（第 2 章提到的传统做法的代表）。互操作（interoperability）：不同工具之间能互相配合工作。Fullspace 的互操作被作者称为“承重墙”——意思是它是 Fullspace 能替代而非并列于 LangGraph 的关键支撑。

10.2 把别人的子流程，当成一本书

`as_capability` 把一个已经编译好的 `LangGraph` 子流程，变成 Fullspace 图书馆里的一本书：

```
cap, handler = as_capability(
    已编译的子流程,
    capability_id="retriever",          # 这本书叫啥
    description="retrieve and summarize documents", # 描述（贴标签用）
    goto="writer",                     # 跑完跳到 writer 这本书
    map_in=lambda s: {"query": s.get("q")}, # 输入怎么对接
    map_out=lambda out, s: {"docs": out["summaries"]}, # 输出怎么对接
)
m.register(cap); eng.bind("retriever", handler)
```

清单 10-1 `as_capability`: 把 `LangGraph` 子图变成一本书

把子流程当终点

如果你既不给 `intent`、也不给 `goto`，引擎就认为“这本书翻完，任务结束”——这就是把某个子流程当成“最终步骤”的写法。

10.3 把自己变成别人的一个零件

反过来，`as_langgraph_node` 把 Fullspace 引擎变成 `LangGraph` 图里的一个节点。对用 `LangGraph` 的人来说，这就是个普通节点，背后其实是整个 Fullspace 图书馆——完全透明。

```
node = as_langgraph_node(
    engine,
    task=lambda s: s["task"],          # 从对方状态里取任务
    map_state_out=lambda fs, lg: {"n": fs["n"]},
)
# 然后像普通 LangGraph 节点一样用
g.add_node("fs_step", node)
```

清单 10-2 `as_langgraph_node`: 把自己嵌入 `LangGraph`

10.4 变成通用积木：Runnable

FullspaceRunnable 让引擎能塞进任何接受 langchain “积木” 的地方（比如 LangChain 的链式表达式、LangServe 服务）。它提供 `invoke/stream/ainvoke/astream` 四个方法，和 LangGraph 编译产物一模一样的接口。

接口对称，所以无缝

引擎本来就自带 `run/stream/ainvoke/astream` 四种启动方式，FullspaceRunnable 只是把它们包装成 langchain 积木的标准接口。所以一个 Fullspace 引擎可以直接写进 `引擎 | 别的积木` 这样的链式表达式，和原生积木没区别。

10.5 小结

互操作是 Fullspace 能 “替代而非并列” 的关键：`as_capability` 把别人的子流程变成一本书（别的→Fullspace），`as_langgraph_node` 把自己变成别人的节点（Fullspace→别的），FullspaceRunnable 让自己变成通用积木。三者都不用改引擎代码。下一章看怎么 “边跑边看进度”。

第 11 章

边跑边看进度：流式与异步

有时候你不想等程序全跑完，想每翻一本书就看到一次结果。这就是流式。这一章讲怎么一步步看进度，以及怎么用异步配合真实的大模型调用。

11.1 每翻一本书，报告一次

用 stream 代替 run，引擎每翻完一本书，就会**立刻给你一个事件（StepEvent）**，告诉你这一步翻的是哪本、记事本变成了啥、还没结束。这样你就能在界面上实时显示进度，而不是干等。

```
StepEvent(  
    step=1,                # 第几步  
    group=["search"],      # 这步翻了哪本/哪几本  
    updates={"found": "..."}, # 这步记了啥  
    state={...},           # 记事本当前全貌  
    trajectory=["search"], # 到现在翻过哪些  
    terminated=False,      # 结束没  
)
```

清单 11-1 StepEvent 里有什么

11.2 同步地看进度

```
for ev in eng.stream("plan the research steps"):
    print(f"第 {ev.step} 步: 翻了 {ev.group}"
          + (f" (结束: {ev.terminated_by}) " if ev.terminated else ""))
```

清单 11-2 同步流式（来自官方示例）

11.3 异步地跑，配合真实大模型

真实产品里，每本书执行时往往要调用大模型或访问网络——这些操作适合用 `async`（异步）来跑，不卡住程序。Fullspace 的 handler 可以直接写成 `async` 函数，引擎会自动 `await` 它。

```
async def search(ctx):                                # 真实场景：这里调大模型/网络
    await asyncio.sleep(0)
    return NodeResult(updates={"found": "facts"},
                          intent="summarize the findings")

async for ev in eng.astream("plan the research steps"):
    print(f"第 {ev.step} 步: {ev.state.get('answer')!r}")
```

清单 11-3 异步节点：为真实大模型调用预留

同一个函数，同步异步都能用

引擎会自动判断 handler 的返回值“要不要等”。所以你写的同一个 handler，既能在同步的 `run` 里用，也能在异步的 `astream` 里用——不用为异步专门改写。这是为了方便你混用各种大模型 SDK（有的同步、有的异步）。

11.4 小结

`stream/astream` 每翻一本书报告一次（`StepEvent`），`run/ainvoke` 是它们的“一口气跑完”版。handler 可以写成 `async`，引擎自动适配。这让你

既能看进度，又能顺畅对接真实大模型。下一章是全书的高潮——用四个真实场景把前面学的全用上。

第 12 章

场景实战：把 Fullspace 用到真实业务里

前面学了原理，这一章把它们用起来。我们用四个真实业务场景——智能客服、文档问答、多步推理、动态扩展——演示 Fullspace 怎么落地。每个场景都给完整可运行代码，并和传统做法对比。

12.1 为什么要看场景

学原理容易，用到自己业务里难。这一章每个场景都遵循同样的结构：**业务故事** → **怎么用 Fullspace 建模** → **完整代码** → **运行结果** → **和传统做法对比**。代码全部用默认的 HashEmbedder（不要密钥、不花钱），你可以立刻在电脑上跑出来。真实产品里，把里面的纯函数换成大模型调用即可。

这些代码都能跑

本章所有代码都经过验证，复制到装好 Fullspace 的环境就能运行。为了能离线跑，我们用纯函数模拟了大模型的回答；真实落地时，把 return 里的假回答换成你调用大模型的代码就行，框架部分一行都不用改。

12.2

场景一：智能客服——把用户问题自动分发给对的能力

业务故事：你做一个电商客服。用户的问题五花八门——退款、物流、商品咨询。你希望机器人能**根据问题的意思**，自动找到对的部门来回答，而不是让用户在一堆按钮里选。

用 Fullspace 怎么想：每个部门 = 一本书（退款、物流、商品）。用户的问题变成数字，找最像的那本书，就是“找对部门”。完全不用画“如果...就...”的判断分支。

```
from fullspace import Capability, HashEmbedder, Manifold
from fullspace.engine import Engine, NodeResult

m = Manifold(HashEmbedder())
m.register_many([
    Capability("refund", "process the customer refund request"),
    Capability("shipping", "track shipping and delivery status"),
    Capability("product", "answer product information and specifications"),
    Capability("end", "wrap up and send the final reply", metadata={"sink": True}),
])
e = Engine(m)
# 每个部门处理完，统一去“结束”这本书汇总回复
e.bind("refund", lambda c: NodeResult(updates={"reply": "退款已受理, 3 个工作日到账"}, goto="end"))
e.bind("shipping", lambda c: NodeResult(updates={"reply": "您的包裹预计明天送达"}, goto="end"))
e.bind("product", lambda c: NodeResult(updates={"reply": "这款手机支持 5G, 电池 5000mAh"}, goto="end"))
e.bind("end", lambda c: NodeResult(updates={"answer": c.state.get("reply", "")}))

# 问题措辞和能力描述有共享词时，字面匹配就能对上
for q in ["I want to process my refund request",
          "track my shipping and delivery status",
          "tell me product information and specs"]:
    r = e.run(q)
    print(r.trajectory, "->", r.state["answer"])
```

清单 12-1 场景一：智能客服意图分发（可直接运行）

和传统做法对比

传统做法要写一堆 if/elif 或训练一个意图分类器，每加一个部门就要改判断代码。这里你只要 register 一本新书 + bind 一个 handler，完全不用动路由逻辑——加部门就像图书馆上新书一样简单。

运行结果（实测）

退款类 -> ['refund', 'end']; 物流类 -> ['shipping', 'end']; 商品类 -> ['product', 'end'], 三种问题各走各的路。注意：这里问题措辞和能力描述故意保留了共享词（refund、shipping、product），所以默认的字面匹配（HashEmbedder）也能对上。真实产品换成语义嵌入后，即使用户换种说法（如“我想退货”“包裹到哪了”），也能正确路由——这正是语义嵌入强于字面匹配的地方。

12.3 场景二：RAG

文档问答——查资料、再总结、再回答

业务故事：做一个“企业知识库问答”。用户提问，程序先去知识库里检索相关文档，再把这些文档综合成答案。这是当下最火的 AI 应用模式之一（叫 RAG）。

用 Fullspace 怎么想：三本书排成一条线——检索 → 综合 → 回答。检索完用 intent（软走向）说“我想综合”，自动找到“综合”这本书。

```

m = Manifold(HashEmbedder())
m.register_many([
    Capability("retrieve", "search and retrieve documents from knowledge base"),
    Capability("synthesize", "read retrieved documents and synthesize an answer"),
    Capability("end", "output the final answer", metadata={"sink": True}),
])
e = Engine(m)
# 检索：用 intent 软走向到“综合”（靠找最像，不画箭头）
e.bind("retrieve", lambda c: NodeResult(
    updates={"docs": "[文档1: 退款政策; 文档2: 7天无理由]"},
    intent="read retrieved documents and synthesize an answer"))
# 综合：把文档变成答案草稿，硬走向到“结束”
e.bind("synthesize", lambda c: NodeResult(
    updates={"draft": "根据文档: 商品支持 7 天无理由退款"}, goto="end"))
e.bind("end", lambda c: NodeResult(updates={"answer": c.state.get("draft")}))

r = e.run("search documents and answer from knowledge base")
print(r.trajectory)          # ['retrieve', 'synthesize', 'end']
print(r.state["answer"])     # 根据文档: 商品支持 7 天无理由退款

```

清单 12-2 场景二：RAG 文档问答（可直接运行）

和传统做法对比

传统 RAG 要手动串起“检索函数→综合函数→输出”的调用顺序。

Fullspace 里，retrieve 不需要知道“下一个是谁”，只要用一句 intent 描述意图，引擎自动找到 synthesize。这让每一步都解耦——你想在中间插一个“事实核查”步骤，只要 register 一本新书，不用改 retrieve 的代码。

12.4 场景三：ReAct

多步推理——想想、做做、看看，循环到解决

业务故事：复杂问题一次搞不定，得像人一样：先**想**该干嘛，再**做**（调工具），再看**结果**，不够就再想……直到解决。这种“思考-行动-观察”的循环，就是大名鼎鼎的 **ReAct** 模式。

用 Fullspace 怎么想：四本书——想(think)、做(act)、看(observe)、结束。think 用 intent 走到 act, act 用 intent 走到 observe, observe 看情况：信息够了就 goto 结束，不够就 intent 回到 think。循环自然形成，不用画回环箭头。

```
m = Manifold(HashEmbedder())
m.register_many([
    Capability("think", "think reason about the problem and plan next action"),
    Capability("act", "act use a tool to gather information or compute"),
    Capability("observe", "observe the result of the action"),
    Capability("end", "final answer output", metadata={"sink": True}),
])

def think(ctx):
    n = ctx.state.get("rounds", 0) + 1
    return NodeResult(updates={"rounds": n},
                      intent="act use a tool to gather information") # -> act

def act(ctx):
    return NodeResult(updates={"tool": f"tool_{ctx.state['rounds']}"},
                      intent="observe the result of the action") # -> observe

def observe(ctx):
    if ctx.state.get("rounds", 0) >= 2: # 想了 2 轮，信息够了
        return NodeResult(updates={"obs": "信息够了"}, goto="end") # -> 结束
    return NodeResult(updates={"obs": "还需更多信息"},
                      intent="think reason about the problem") # -> 回 think

e = Engine(m)
e.bind_many({"think": think, "act": act, "observe": observe,
            "end": lambda c: NodeResult(updates={"answer": "问题已解决"})})

r = e.run("think and solve the problem step by step")
print(r.trajectory) # ['think', 'act', 'observe', 'think', 'act', 'observe', 'end']
```

清单 12-3 场景三：ReAct 多步推理（可直接运行）

和传统做法对比

传统框架画 ReAct 循环，要显式连 think→act→observe→think 的回环箭头，还要在 observe 处画条件边判断“回不回 think”。Fullspace 里这些都不用画——走向靠 intent 自动找，回环靠 observe 发一句“我还想再想”的 intent 自然形成。代码读起来就是业务逻辑本身。

运行结果（实测）

轨迹是 think→act→observe→think→act→observe→end，正好两轮思考后判定信息够、走向结束。在真实场景里，think/act 里调用大模型和真实工具，逻辑骨架完全一样。

12.5 场景四：遇到没见过的需求，自动“长”出能力

业务故事：你不可能预先想到用户所有需求。当用户提了个你的能力库完全没有的需求时，传统系统会报错或答非所问。Fullspace 可以**当场造一个新能力**加进库里——这就是第 8 章讲的“物化”。

用 Fullspace 怎么想：给路由器配一个“造书匠”（materializer）。当现有书都不够像时，造书匠现场印一本新书上架，然后执行它。

```

from fullspace.engine.router import Router

m = Manifold(HashEmbedder())
m.register(Capability("greet", "greet the user say hello"))  # 只有一本“打招呼”

def 造书匠(描述, 分数):
    # 真实场景：这里可以调用大模型，根据描述生成一个新能力的处理逻辑
    print(f" [系统] 现场造了一本新书：{描述}")
    return Capability("translator", 描述)

e = Engine(m, router=Router(
    m, threshold=0.99,          # 及格线设极高，强制触发“找不到”
    materializer=造书匠,
))
e.bind("greet", lambda c: NodeResult(intent="please translate this to another language"))
e.bind("translator", lambda c: NodeResult(updates={"result": "翻译完成"}, halt=True))

r = e.run("greet the user say hello")
print(r.trajectory)          # ['greet', 'translator']
print("translator" in m)    # True, 新书已被造出并上架

```

清单 12-4 场景四：动态能力涌现（可直接运行）

和传统做法对比

传统框架的流程图是上线前画死的，运行时不可能加节点。Fullspace 的图书馆能在运行时自己生长——这适合那些“需求会随时间演化”的产品。当然，造新书要谨慎（真实场景里造书匠通常会接大模型，并加人工审核），但这个能力本身就是传统框架给不了的。

运行结果（实测）

greet 执行后发出“翻译”意图，库里没有翻译能力（分数不够），触发物化：现场造出 translator 并上架，接着执行它，halt 结束。轨迹 ['greet', 'translator']。

12.6 场景串讲：这些场景背后的共同点

回看四个场景，你会发现它们用的是**同一套零件**（Capability、Engine、NodeResult、Router），只是组合方式不同。这就是 Fullspace 的价值：你不用为每种业务学一套新东西，只要会“登记书 + 绑定处理函数 + 说清走向”，就能搭出客服、问答、推理、动态扩展等各种系统。

场景	主要用到的机制	走向方式
智能客服	软路由分发	起点靠 task 文本找最像
RAG 问答	线性多步	intent 软走向串起
ReAct 推理	循环 + 条件退出	intent 循环 + goto 退出
动态扩展	物化涌现	materializer 造新书

表 12-1 四个场景用到的核心机制

12.7 小结

四个场景演示了 Fullspace 落地真实业务：客服靠软路由分发、RAG 靠 intent 串步骤、ReAct 靠 intent 循环+goto 退出、动态扩展靠 materializer 造新书。它们共享同一套零件，只是组合不同。把场景里的纯函数换成大模型调用，就是生产级应用。下一章我们用数据看看 Fullspace 到底比传统做法强多少。

第 13 章

到底强多少：和真实 LangGraph 的对比

光说“好”没用，得测。这一章讲 Fullspace 自带的评测工具怎么和真实的 LangGraph 对比，以及怎么老实读这份结果——包括 Fullspace 在某些地方其实是“输”的。

13.1 怎么对比才公平

Fullspace 自带一个评测工具（eval），在相同任务上，把它和**你电脑上真实安装的 LangGraph** 直接对比，看五个指标：是否答对、节点执行了几次、路由判断了几次、花了多少时间、两次运行结果是否一致。

指标	看什么
是否答对	有没有产出预期结果
节点执行次数	翻了几本书（少更好）
路由判断次数	决定走向几次（少更好）
耗时	花了多久（仅供参考）
是否可复现	同样的输入，结果一不一样

表 13-1 评测看五个指标

13.2 六个对比任务

评测准备了六个任务，前四个是传统做法也能做的（线性、分支、循环、ReAct），后两个是**只有 Fullspace 能做的**（运行时造新书、处理没见过的问题）。

任务	内容	传统做法能做吗
线性	$A \rightarrow B \rightarrow C$	能
分支	按问题选入口	能（条件边）
循环	循环几次再结束	能（条件边回环）
ReAct	思考-行动-观察循环	能
动态造能力	运行时新增能力	不能
处理没见过的问题	OOD 鲁棒性	不能（会报错）

表 13-2 六个对比任务

13.3 老实说：简单任务上，Fullspace 反而多干活

在那些简单的、固定不变的任务上（线性、分支等），Fullspace 和 LangGraph **答得一样对**。但要老实承认一点：LangGraph 因为提前画好了

箭头，判断走向是“免费”的；Fullspace 每翻一本书要多做一次“找最像”。所以在这些**简单静态**任务上，Fullspace 的路由判断次数其实更多。

Fullspace 不是处处都赢

在简单、固定、不会变的任务上，传统做法的预画箭头反而更省。Fullspace 的优势在“动态、会变、规模大”的场景。选工具要看你的任务长什么样，别盲目。

13.4 能力很多时：Fullspace 大幅领先

当能力数量变大（几千到几万），情况就反过来了。Fullspace 用 FAISS 的快速查找，路由延迟比传统做法快约 **80 到 120 倍**。这是因为它能“次线性”地查找，而传统做法随着能力变多越来越慢。

这份评测为什么可信

它有五个“老实”的信号：(1) 同样的数据和问题，只换查找方法这一变量；(2) 用固定随机种子，结果可复现；(3) 计时前先热身，排除冷启动；(4) 老实承认数据少时两种方法其实差不多；(5) 结论分级——只有数据够多时才敢说“快几十倍”。这种“承认劣势→指出翻转条件→用受控实验佐证”的写法，比光喊口号可信得多。

13.5 怎么自己跑这份评测

```
pip install langgraph
python -m fullspace.eval          # 六任务对比表
python -m fullspace.eval.scaling  # 能力变多时的延迟对比
```

清单 13-1 跑评测（需要先装 langgraph）

评测结果是“事实来源”——在宣称任何“Fullspace 更强”之前，先自己跑一遍。这也是 Fullspace 作者反复强调的态度。

13.6 小结

评测工具拿 Fullspace 和真实 LangGraph 对比。简单静态任务上 Fullspace 路由反而更多（老实承认）；动态和 OOD 任务上 Fullspace 赢（传统做不了）；能力很多时 Fullspace 快 80~120 倍。读结果要先自己跑、再下结论。最后一章讲怎么给 Fullspace 加自己的部件。

第 14 章

给 Fullspace 加你自己的部件

Fullspace 的每个核心部件都可以替换成你自己的实现。这一章用四个最小例子，演示怎么换嵌入、换查找、换翻书节奏、换存档方式。

14.1 换一种“文字变数字”的方法

只要写一个类，实现 `embed` 方法（输入一句话，返回一串数字），就能当嵌入器用。下面是个用固定随机投影的演示（生产请接真实模型）：

```
from fullspace.manifold.embedding import Embedder
import numpy as np, re

class MyEmbedder(Embedder):
    def __init__(self, dim=128):
        self.dim = dim
        rng = np.random.default_rng(0) # 固定种子, 保证可复现
        self._P = rng.standard_normal((dim, 4096)).astype(np.float32)
    def embed(self, text):
        v = np.zeros(4096, dtype=np.float32)
        for tok in re.findall(r"[a-z0-9]+", text.lower()):
            v[hash(tok) % 4096] += 1.0
        v = self._P @ v
        n = np.linalg.norm(v)
        return v / n if n > 0 else v

m = Manifold(MyEmbedder(128))
```

清单 14-1 自定义嵌入器

务必用固定种子

嵌入器生成的数字必须每次一样（否则同一句话每次数字不同，图书馆就乱了）。所以随机投影的矩阵要用 `default_rng(0)` 这种固定种子生成——这也是默认 `HashEmbedder` 用 `md5` 的原因。

14.2 换一种查找方法

实现 `add/search/remove` 等方法，就能当查找索引。比如接一个更快的近似查找库（如 `hnswlib`）：

```
from fullspace.manifold.index import AnnIndex

class HnswIndex(AnnIndex):
    def __init__(self, dim):
        import hnswlib
        self.dim = dim
        self._idx = hnswlib.Index(space="cosine", dim=dim)
        self._ids = []
    def add(self, id, vector): ...      # 维护 id 与内部编号的映射
    def search(self, query, k=5): ...   # 返回 [(id, 分数), ...]
    def remove(self, id): ...
    def vector_of(self, id): ...
    def __len__(self): return len(self._ids)
```

清单 14-2 自定义查找索引（接 hnswlib 示意）

14.3 换一种“翻书节奏”

实现 select 方法（输入图书馆和需求，返回要翻的几本书），就能当流动策略用。比如“每步翻最像的两本”：

```
from fullspace.engine.flow import FlowPolicy

class TopTwoFlow(FlowPolicy):
    def select(self, manifold, query):
        return manifold.nearest(query, k=2)

eng = Engine(m, flow=TopTwoFlow())
```

清单 14-3 自定义流动策略

14.4 换一种存档方式

实现 put/get/list 三个方法，就能当检查点器用。比如接 Redis（一个高速缓存服务）：

```
from fullspace.state import Checkpointer, Checkpoint

class RedisCheckpointer(Checkpointer):
    def __init__(self, client): self._r = client
    def put(self, cp): ...      # 存一条检查点
    def get(self, thread_id, checkpoint_id=None): ...  # 不给 id 就取最新
    def list(self, thread_id): ...  # 按时间顺序返回全部
```

清单 14-4 自定义检查点器（接 Redis 示意）

14.5 小结

Fullspace 的四个核心部件——嵌入器、查找索引、流动策略、检查点器——都只要写一个类、实现几个方法就能替换。你可以从默认的“够用”配置，平滑升级到生产级的神经嵌入、FAISS/HNSW 查找、Redis 存档，业务代码不用改。到这里，你已经能驾驭整个 Fullspace 了。结合第 12 章的场景实战，去搭你自己的智能体吧。

附录 A

示例索引：能直接跑的例子

Fullspace 自带的示例，每个演示一种用法，都能直接运行。

示例	演示什么	运行命令
linear_pipeline	A→B→C 一条线（最基本）	<code>python -m fullspace.examples.linear_pipeline</code>
branching	按问题意思选不同入口	<code>python -m fullspace.examples.branching</code>
react_agent	思考-行动-观察循环	<code>python -m fullspace.examples.react_agent</code>
interrupt_resume	存档、断点续跑	<code>python -m fullspace.examples.interrupt_resume</code>
streaming	边跑边看进度	<code>python -m fullspace.examples.streaming</code>

表 A-1 官方示例与命令

另外：python -m fullspace.viz 生成 3D 能力球；python -m fullspace.eval 跑对比评测；python -m fullspace.eval.scaling 跑规模延迟对比。后两个需要额外装 langgraph / faiss-cpu。

附录 B

API 速查

最常用的类和方法（基于 0.1.0）。第一次看不懂没关系，结合正文回顾。

```
Capability(id, description, metadata={}, vector=None)
Manifold(embedder, index=None, projector=None)
    .register(cap) / .register_many(caps) / .remove(id)
    .nearest(query, k=5) -> list[Hit]      # query 可以是文字或数字
    .find_or_materialize(query, threshold=0.5, k=1, materializer=None)
    .get(id) / .project(id) / .project_all()
HashEmbedder(dim=256); CachedEmbedder(inner, maxsize=4096)
NumpyAnnIndex(dim); FaissIndex(dim, nlist=100, nprobe=10)
```

清单 B-1 manifold (图书馆)

```
Engine(manifold, flow=None, router=None, max_steps=None,
       state_spec=None, checkpointer=None)
    .bind(id, handler) / .bind_many({id: handler})
    .run(task, state=None, thread_id=None, max_steps=None) -> RunResult
    .stream(...) -> 每步一个 StepEvent
    async .ainvoke(...) / async .astream(...)
    .resume(thread_id, task) / .history(thread_id) / .get_checkpoint(thread_id)
NodeResult(updates={}, intent=None, goto=None, halt=False)
Router(manifold, threshold=0.3, margin=0.15, disambiguator=None, materializer=None)
DiscreteFlow(); FieldFlow(width=3); WavefrontFlow(base_width=2)
```

清单 B-2 engine (管理员)

```
merge_updates(state, updates, spec=None)
overwrite / last_value / add          # 三种合并规则
InMemoryCheckpoint(); SqliteCheckpoint(path=None)
as_capability(app, id, desc, *, intent=None, goto=None, map_in=None, map_out=None)
as_langgraph_node(engine, task, *, map_state_out=None)
FullspaceRunnable(engine)
```

清单 B-3 state (记事本) 与 interop (打通)

附录 C

术语表（大白话版）

把全书的关键术语集中在这里，每条都配一句大白话解释。遇到不懂的词，先来这查。

术语	大白话解释
智能体 / agent	能自己分好几步完成任务的 AI 程序
LLM / 大语言模型	像 ChatGPT 那样能理解和生成文字的 AI 大脑
向量 / vector	代表一句话“含义”的一串数字（如 256 个数）
嵌入 / embedding	把文字变成向量的过程（给书贴数字标签）
最近邻	数字最像的那一个；找它就叫检索
ANN	书很多时快速“近似”找最像的方法，比一本本比快
余弦相似度 / cosine	算两串数字有多像的分数，越大越像（-1 到 1）
能力 / Capability	图书馆里的一本书，能干的一件事
流形 / manifold	那家“图书馆”，所有能力和数字标签住的地方
sink / 汇点	标了 sink=True 的能力，翻到它就结束
intent / 意图	“接下来想干什么”的一句话，用来软走向（找最像）
goto	直接点名跳到某个能力（硬走向）
halt	强制立刻停止
轨迹 / trajectory	依次翻过哪些书的顺序记录
状态 / state	那个记事本，存着到目前为止的所有结果
reducer	记事本某字段“怎么合并新值”的规则（覆盖/累加/有才换）
路由器 / Router	决定“下一步翻哪本”的大脑

术语	大白话解释
阈值 / threshold	“够不够像”的及格线，默认 0.3
亲和力裁剪	最像的够及格线就直接用，不反复判断
物化 / materialize	找不到够像的就现场造一本新书（涌现）
消歧器 / disambiguator	前两名太接近时请的外援（常接大模型）
流动策略 / FlowPolicy	每步翻几本书的节奏（一本/一批/越翻越多）
检查点 / Checkpoint	游戏存档：记事本+走到第几步+翻过哪些书
thread_id	这次会话/任务的名字，给了才自动存档
断点续跑 / resume	从最近存档接着跑
PCA / 投影	把高维数字压成 3 维给人看，程序判断不用它
互操作 / interop	和别的工具（LangGraph 等）互相打通
承重墙	互操作：移除不影响引擎，有则接入整个生态
LangGraph	最流行的“画流程图”式智能体框架
RAG	检索增强问答：先查资料再综合再回答
ReAct	思考-行动-观察循环的多步推理模式
OOD	分布外：没见过/没预料到的输入
pip / PyPI	Python 的安装工具 / 官方包仓库

表 C-1 术语表

附录 D

进阶实战：混合路由——让 Fullspace 全维度不输

这一附录讲一个实测验证过的优化技巧：让确定的流程走 goto、语义分叉走 intent、再缓存重复决策。它能让 Fullspace 在路由开销上追平传统图，同时保留动态能力。数据来自一个 8 个 agent 的 K12 教育双框架对比实验。

D.1 一个浪费：纯 intent 每跳都做一次 ANN

第 8 章讲过，Fullspace 默认用 intent 软路由，每翻一本书都要做一次「找最像的」（ANN）。这在需要语义判断的地方很值，但在**完全确定的流程段**上就浪费了——比如 diagnose 之后一定去 plan，这种固定走向根本不用算。

用传统图（LangGraph）的人常拿这点质疑 Fullspace：「你每跳都算一次，我静态边 0 次计算，你凭什么赢？」其实这不是 Fullspace 的本质劣势，而是**默认策略偏保守**——它默认全程用语义路由，是为了最大化灵活性。一旦你明确某些段是确定的，就可以省掉那些 ANN。

D.2 混合路由：确定走 goto，语义走 intent

优化思路很简单：**确定的流程段用 goto (0 次 ANN)**，**只在真正需要语义判断的分叉用 intent**。goto 是硬走向（第 5 章），跳转免费；intent 是软走向，留给不确定的地方。

```
# 线性段：diagnose 之后一定去 plan，用 goto (0 次 ANN)
def diagnose_h(ctx):
    return NodeResult(updates=diagnose(ctx.state), goto="plan")

# 分叉处：grade 后去 report 还是 analyze，用 intent (1 次 ANN)
def grade_h(ctx):
    updates = grade(ctx.state)
    wrong = {**ctx.state, **updates}.get("wrong_count", 0)
    intent = ("report summarize learning outcomes"
             if wrong == 0 else "analyze diagnose mistakes")
    return NodeResult(updates=updates, intent=intent)
```

清单 D-1 混合路由：线性段 goto，分叉 intent

一个原则：把 ANN 留给真正不确定的地方

goto 负责「我知道下一步是谁」的确定跳转（免费）；intent 负责「我得想想下一步该干嘛」的语义跳转（1 次 ANN）。两者搭配，既保留语义路由能力，又不为确定流程付冤枉钱。

D.3 再加一层：决策缓存

有些场景里，同一个 intent 会反复出现（比如 ReAct 循环里老说「act」「observe」）。第一次算出结果后把它记下来，下次同一个 intent 直接复用，跳过 ANN。


```
class HybridRouter(Router):
    def route(self, intent):
        if intent in self._cache:          # 命中缓存
            cap = self.manifold.get(self._cache[intent])
            return Hit(cap, 1.0)           # 0 次 ANN
        decision = super().route(intent)   # 真正走 ANN
        self._cache[intent] = decision.capability.id
        return decision
```

清单 D-2 决策缓存：重复 intent 跳过 ANN

为什么缓存绝对正确

「把一句话变成数字再找最近邻」是确定的过程（同样的话永远得到同样结果），所以缓存永远不会失效。循环场景下，缓存能让昂贵的 ANN 调用数大幅下降。

D.4 实测：8 个 agent 的 K12 教育对比

为了验证这套优化到底有没有用，作者搭了一个 K12 教育 demo：8 个教学 agent（学情诊断、学习规划、讲解、出题、批改、错题分析、答疑、报告），分别用 LangGraph、纯 Fullspace（全程 intent）、混合版 Fullspace（goto + intent + 缓存）三套方式实现，跑 200 个随机学生负载。结果：

指标	LangGraph	Fullspace(纯intent)	Fullspace-混合
功能正确性	三版 100% 一致	三版 100% 一致	三版 100% 一致
路由开销/回环场景	4 次条件边	9 次 ANN	4 次 ANN（追平）
延迟中位（普适）	2.30 ms	0.36 ms	0.15 ms（最快）
运行时可加 agent	需重新编译	可以	可以

表 D-1 三版对比（K12 教育 8 agent，200 个随机负载）

关键结论

混合版在「路由开销」上追平了 LangGraph（回环场景都是 4 次），延迟反而最低（0.15ms），同时仍能运行时加 agent。这证明：**Fullspace 的「路由次数多」不是本质劣势，是默认策略选择**，用混合路由就能补齐。

D.5 下一步：让框架自己学

这个 demo 里，goto/intent 是开发者手动决定的。在产品级，可以让框架**自动学习**：发现某个 intent 连续多次路由到同一个能力，就自动把它「固化」成 goto（路由热点学习）。这样小白什么都不用管，框架在「确定处省 ANN、动态处用语义」之间自动切换——这才是「自适应混合路由」的完整形态。

完整的 demo 代码和可视化报告在仓库的 `demos/k12-education/` 目录下，包含三版实现、200 负载测试、规模 scaling 曲线和交互式报告，你可以直接跑起来验证这些数字。